czii-cryo-et-object-identification

# CZII - CryoET Object Identification コンペ 8位解法 ＋コード公開

| | |
|---|---|
| **Rank** | 8 |
| **Team** | I Cryo Everyteim |
| **Author** | Sergio Alvarez |
| **Topic ID** | 561515 |
| **Version** | Japanese Translation |

Source: https://www.kaggle.com/competitions/czii-cryo-et-object-identification/discussion/561515
Retrieved with Kaggle CLI on 2026-10-21.

Kaggle Discussionの主投稿と、技術的補足を含む選定済みQ&Aを日本語へ翻訳した版です。code、URL、model名、識別子、数値は原文を保持しています。

まず、このコンペを開催してくださったコンペ主催者と Kaggle スタッフに感謝します。以下では、チーム I Cryo Everyteim（@sirapoabchaikunsaeng、@iamparadox、@itsuki9180、@sersasj）の解法を紹介します。

## 背景

- ビジネス上の背景: コンペの概要
- データの背景: コンペのデータ

## アプローチの概要

最終提出は4つの 3D U-Net のモデルスープからなり、モデル同士がよく補い合うように、異なるモデルサイズ、パラメータ、学習データで学習しました。すべての 3D U-Net はパッチサイズ (128, 128, 128) で学習しましたが、推論はパッチサイズ (160, 384, 384) で行い、境界のアーティファクトに対処するため、25% のオーバーラップと gaussian reconstruction を使いました。さらに、反転と転置を含む幾何的な test-time augmentation (TTA) を使いました。

## モデル

私たちのモデルは、もともと主催者のサンプル notebook と @fnands の notebook に基づいています。MONAI ライブラリの 3D U-Net アーキテクチャを使い、パッチサイズ (128, 128, 128) で学習しました。

モデルは3レベルの深さで、strides は (2, 2, 1) です。最初は単純なモデルから始めました。

```
:
:
:
```

この単体モデルは、私たちの推論戦略と組み合わせて public leaderboard のスコア 0.744 に達しました。

その後、次の設定で、より複雑な U-Net を学習しました。

```
:,
:,
:,
```

上の設定のモデルは単独で LB 0.759 のスコアを達成しました。

```
:,
:,
:,
```

モデルには 0.2 または 0.3 の dropout も適用しました。

# 学習

モデルは、Gaussian denoising をかけた6つの合成トモグラム、具体的には
'TS_0'、'TS_1'、'TS_10'、'TS_11'、'TS_12'、'TS_13' のトモグラムで事前学習しました。

@sersasj の注: Gaussian denoising を適用したのは、WBP トモグラムでの粒子の見え方が視覚的に改善し、実装も簡単だったからです。より高度な denoiser を使えばもっと良い結果が得られると考えましたが、denoising 用のモデルをコーディングしようとしたら Kaggle のクォータを使いすぎたので、あきらめました。

事前学習は、モデルが粒子を学習するのに必要な時間を減らしただけでなく、LB スコアもおよそ 0.01 上げました。

合成データでの事前学習とファインチューニングの両方で、次の変換を使っています。

```
Compose([
   RandCropByLabelClassesd(
       keys=[, ],
    =,
       spatial_size=[128, 128, 128],
       =7,  # background,  all 6 classes
    =16,
  ),
   RandFlipd(keys=[, ], =0.5, =0),
   RandFlipd(keys=[, ], =0.5, =1),
   RandFlipd(keys=[, ], =0.5, =2),
])
```

MONAI の DiceCELoss を使い、AdamW でモデルを最適化しました。学習率は plateau で減らし、初期学習率は 1e-3 です。

Exponential Moving Average (EMA) も試し、良い結果が出ました。コンペの最後の3日間で、他のトモグラムの種類 "denoised"、"ctfdeconvolved"、"isonetcorrected" を使ってモデルを学習しました。コンペのほぼ全期間を通じて、学習時の幾何的なデータ拡張（反転と回転）以外に LB を上げる戦略は見つかっていませんでしたが、他のトモグラムの種類での学習は良い結果をもたらしました。

# 検証戦略

主に k-fold モデルの out-of-fold 予測に頼りました。具体的には、1つを除くすべてのトモグラフィーで学習し、残りの1つを検証セットに使う 7-fold の交差検証を実装しました。これを7回繰り返し、各 fold が1回ずつ検証セットになるようにしました。これにより、public leaderboard のスコアとよく相関するスコアが得られました。通常は public より 0.01 から 0.02 ポイント高い値でした。

一つの限界は、モデルスープのアンサンブルを試すときには、この検証戦略を適用できなかったことです。その場合は、リークしたデータのスコアに頼らざるを得ませんでした。このアプローチは leaderboard のスコアとそれほど強くは相関しませんでしたが、ほとんどの場合、異なるアンサンブルの性能を順位づけし、TTA 戦略を比較することはできました。

# 提出の詳細

## 推論

推論では、境界の予測をより正確に扱うため、25% のオーバーラップと Gaussian reconstruction を使った MONAI の sliding_window_inference を使います。これで public LB が約 0.002 上がりました。最大の発見の一つは、できるだけ大きなパッチで推論することでした。ウィンドウサイズ (160, 384, 384) を使い、サイズ (128, 128, 128) と比べてスコアが約 0.01 上がりました。（これを見つけてくれた @iamparadox に感謝します。）

異なるサイズの予測を組み合わせること、たとえば (160, 384, 384) + (136, 248, 248) も試しましたが、推論にずっと長くかかる一方で、得られる利得ははっきりしませんでした。そのため、このアプローチはやめて、より多くのアンサンブルを作り、より多くの TTA を使う方を選びました。

後処理では、確率を直接平均するのではなく、logit のレベルでモデルの予測を組み合わせることで、粒子検出を改善しました。

その後、watershed セグメンテーションで粒子の特定を洗練させました。これは、接触したり重なったりしている粒子を分離するのに役立ちました。watershed の処理では、粒子ごとの確信度の閾値で二値マスクを作り、距離変換を計算して粒子間の分離を測り、局所最大をマーカーとして個別の粒子を特定し、最後に watershed セグメンテーションを適用して粒子の境界を決めました。推論時間を減らすため、skimage から cucim と cupy に切り替えました。

偽陽性の数を減らすため、粒子ごとに特定の blob のサイズ閾値も使いました。

さらに、X 軸と Y 軸に沿った反転と転置の TTA を使いました。

## アンサンブル

アンサンブルは常に、私たちのチームの主要なアイデアの一つでした。締め切りの2週間前にチームが完全に揃ったとき、私たちは全員 LB スコア 0.73 前後、leaderboard で約50位でした。当時、私（@sersasj）は 0.682 のスコアの YOLO と 0.692 のスコアの U-Net を持っていて、それらをアンサンブルすると LB で 0.732 に達しました。チームの残りのメンバーは 0.739 に達した 3D U-Net しか持っていませんでした。なので、すべてをアンサンブルしてすばらしい結果を出すのは当然だと思っていましたが、残念ながらうまくいきませんでした。

それでも、この経験から貴重な教訓を学びました。私の 3D U-Net は 0.739 のスコアの U-Net より約30倍小さかったのですが、チームのすばらしい推論手法（より大きなパッチ、blob の閾値、logit レベルでのアンサンブル、watershed 処理）と組み合わせると、非常に効果的であることが分かりました。事前学習と追加の学習を実装した後、leaderboard スコア 0.744 に達しました。

チームでの作業（特に @sirapoabchaikunsaeng と @iamparadox）のおかげで、パラメータを変更し、EMA（Exponential Moving Average）を実装し、より多くのデータを取り込み、上記の手法を適用することで、0.744 のモデルから派生したいくつかの U-Net の変種をうまく学習できました。

こうして、互いに補い合うさまざまな U-Net が手元にありましたが、全部を使うことはできませんでした。手持ちのものを全部「使う」ために、@itsuki9180 がモデルスープと呼ばれる手法のアイデアとコードを提示してくれました。これは本質的には、同じ事前学習済みの重みから学習した複数のモデルの重みを平均するものです。

私たちはこれを k-fold 学習のモデルに適用しました。

最終アンサンブルは次のもので構成されていました。

- 次のパラメータで学習した 7 fold のスープ:
  - Channels: (32, 64, 128, 128)
  - residual unit の数: 1
  - 合成データで事前学習
  - EMA なしで denoised の画像で学習
- 次のパラメータで学習した 7 fold のスープ:
  - Channels: (32, 64, 128, 256)
  - residual unit の数: 2
  - 合成データで事前学習
  - EMA なしで、denoised、IsoNet-corrected、CTF-deconvolved のすべての画像で学習
- 次のパラメータで学習した 3 fold（TS_69_2、TS_86_3、TS_99_9）のスープ:
  - Channels: (32, 96, 256, 384)
  - residual unit の数: 2
  - 合成データで事前学習
  - EMA ありで denoised の画像で学習
- 次のパラメータで学習した 7 fold のスープ:
  - Channels: (32, 96, 256, 384)
  - residual unit の数: 2
  - 合成データで事前学習
  - EMA なしで、denoised、IsoNet-corrected、CTF-deconvolved のすべての画像で学習

## 効かなかったこと

- データ拡張を増やす
- CutMix、MixUp
- 2D U-Net のアプローチ（LB は 0.636 にしか達しなかった）
- YOLO とのアンサンブル（最終解法では。約 0.73 の LB に達するまではよく効く）
- より大きく深い U-Net
- cross entropy だけの loss 関数

## 参考資料

- Model Soup の論文
- 主催者のサンプル notebook
- @fnands の notebook

# コード

kaggle czii コンペの8位解法のコード（github）

提出 Notebook

提出 Notebook のクリーン版

# 補足Q&A

## Ma Edward — 質問

おめでとうございます！！！とても示唆に富むアンサンブルの解法です。

最終アンサンブルに含まれる単体モデルの性能を共有していただけますか。

## IAmParadox — 回答

最終アンサンブルの単体モデルの1つは、次の設定でした。

"channels": (32, 64, 128, 256), "strides": (2, 2, 1), "num_res_units": 2,

すべての fold からモデルスープを作り、その public LB は 0.762 でした

## Aurora_blue — 質問

おめでとうございます！

@iamparadox

これは私にとって興味深い点です。

> 最大の発見の一つは、できるだけ大きなパッチで推論することでした。ウィンドウサイズ (160, 384, 384) を使い、サイズ (128, 128, 128) と比べてスコアが約 0.01 上がりました。

モデルは同じパッチサイズで学習したのですか、それとも違いますか。

違う場合、なぜこれでスコアが上がるのか理解できません。

理論的な背景があれば共有していただけるとありがたいです。

## IAmParadox — 回答

モデルは 128x128x128 のパッチサイズでのみ学習しましたが、推論は 160x384x384 のパッチサイズで行いました。正直なところ、このトピックの理論的な背景はあまり知りませんが、ここでの考え方は、推論中に文脈情報が変わらないということです。たとえば 3x3x3 の conv フィルタが、活性化するために入力画像の中心ピクセルに高い値を期待しているとすると、推論時にパッチサイズを大きくしても、それは変わりません。2つ目の理由は、大きなパッチサイズで推論すると境界のアーティファクトが減ることです。

画像を小さい次元にダウンサンプリングしてから高い次元で推論する場合には、この大きなパッチサイズのトリックは効きません。その場合は文脈情報が変わってしまう、つまり、モデルがこれまで見たことのない細部を見ることになるからです。

## Sergio Alvarez — 回答

@clearwaterkzk 大きなパッチによる違いを示すために、最良の提出をパッチ 128 で提出してみました。

最良の提出: public lb: 0.78022, private lb: 0.77612

パッチ 128 の提出: public lb: 0.77464, private lb: 0.76604